

1 Introduction

In this project, you are tasked with designing a hardware accelerator to work in conjunction with the PicoRV32 RISC-V core. Your objective is to perform an FFT on an audio signal by creating two different accelerator designs: a high-performance (only speed matters) version and an energy-efficient version (only energy matters).

1.1 Project requirements

- Fixed core area: $596.4\ \mu\text{m} \times 596.4\ \mu\text{m}$
- Setup & Hold timing reports clean as well as fully DRV clean¹
- Connectivity, Geometry, Antenna reports must be clean
- The Fourier transform of any audio input of a variable number of chunks should be computed correctly (we will perform an automated private test on your project files to test this, public tests are provided)
- The requirements for the two accelerator designs are as follows:
 - For the high-performance design, a latency lower than 61.00 microseconds (baseline latency) is required
 - For the energy-efficient version, an energy lower than 24.6 nJ (baseline accelerator energy) is required.
- You are free to change the clock speed, but a minimum clock speed of 10 MHz is required for the energy-efficient design.

1.2 Important Things for Your Design

- **Design Folder:** Please only make use of source code and scripts in `/data/labs/2026/project`.
- **Task:** to correctly compute the Fourier Transform of a variable-chunk-length audio sample
- **Testbench:** Your design must be compatible with the provided `tb_et4351.sv` testbench, which **MUST NOT** be modified.

¹`max_tran` violations are acceptable. However, keep in mind that when these violations occur, it means that the timing table does not cover the requested input slew and output load combination, and extrapolation has to be performed. Therefore, the static timing analysis using this gate delay may be wrong. This is especially dangerous if the setup slack close to 0.

- **Software:** In your software, you are **MANDATED** to start the accelerator as soon as you run any algorithm on the data (see **!!!! 6.3 Start accelerator**), and you are **REQUIRED** to let the accelerator set the finish flag to indicate when your algorithm is finished, after which no more algorithmic steps are allowed (see **accel_audio.c** for correct usage as well as the simplified example below). This way, the latency calculation in the testbench is correct. Note that you are allowed to move data from the ROM to the DUT (either RISC-V SRAM or your own accelerator) before enabling the acceleration and transmit the FFT'd data through UART after receiving the finish flag. Hence, such data movements do not add towards your run time. However, any data movement during your algorithm **SHOULD** add to your runtime.

```
1 static void accelerated_fft(int n, int chunks, int bits) {
2     // 1. Initialize accelerator
3     ...
4
5     // 2. Reset accelerator
6     ...
7
8     // 3. Pass information to accelerator registers
9     ...
10
11    // 4. Write twiddles to SRAM of accelerator
12    ...
13
14    for (int chunk = 0; chunk < chunks; chunk++) {
15        // 6.1 Reset accelerator
16        ...
17
18        // 6.2 Write input array to SRAM of accelerator
19        ...
20
21        // !!!! 6.3 Start accelerator
22        REG_CONFIG_AND_STATUS |= MASK_CSR_ENABLE;
23
24        // 6.4. Optionally, depending on your implementation, you can:
25        // - Move data in and out of the accelerator for extra acceleration steps
26        // - Perform part of the algorithm on the RISC-V core
27        // - etc.
28
29        // !!!! 6.5 Wait for accelerator to finish
30        while (!(REG_CONFIG_AND_STATUS & MASK_CSR_DONE));
31
32        // 6.6 Disable accelerator
33        REG_CONFIG_AND_STATUS &= ~MASK_CSR_ENABLE;
34
35        // 6.7 Print FFT output for the current chunk
36        ...
37    }
38 }
```

- **Functionality:** Use behavioural simulation for prototyping and latency measurements. Use post-layout simulation without activity annotation for your final functional verification.

- **Real-world application:** By default, `run_all.sh` generates a noisy audio sample stored in `firmware/full_sound.wav` for processing in the behavioural simulation. Subsequently, when `run_all.sh` calls the `verify.py` script, using the output of the accelerator the noise from the audio is removed in the frequency domain. Following this, an audio file is generated by applying an inverse Fourier transform on this data. This recovered audio file is stored under `sim_behav/recovered_audio.wav`. Play both files to hear the differences!
- **Differences between simulation settings:** The noisy audio signal as described above is composed of 24 chunks. However, since the number of samples per chunk is the same for every chunks, the time to process each chunk is the same. Therefore, to save simulation time, only the behavioural simulation performs the FFT of the full audio signal, while the structural and physical simulation only use a single chunk of audio. This also means that in this document, all reported latencies refer to processing a single chunk.
- **Power:** You should do post-layout power estimation (you can use script 11 from the place and route flow for this so that you do not have to rerun your flow every time). The activity file should be generated by measuring the power during accelerated FFT of one audio chunk (see next point). The activity file coverage should achieve 100%.
- **VCD start and duration time:** Make sure to only measure the energy from the FFT acceleration. Therefore, update the VCD extraction scripts with the right start time and duration as reported by the behavioural simulation of your FFT implementation in order to compute power in the right window.

2 Baseline

Below are the specification and performance of the baseline hardware solution.

Chip specification:

- Core area: $596.4\ \mu\text{m} \times 596.4\ \mu\text{m}$
- Clock period: 83.33 ns
- Clock frequency: 12 MHz

Performance:

- Latency in cycles: $N = 732$ cycles
- Latency in seconds: $T = 61.00\ \mu\text{s}$
- Post-layout total power: $P = 0.403\ \text{mW}$
- Approximated energy consumption: $E = P \times T = 24.6\ \text{nJ}$

2.1 Evaluation Criteria

You are required to submit the SystemVerilog/Verilog source code of your design, all the scripts for structural and physical design and all reports. As we will run your final design to verify its correctness, create a separate directory called `finaldesign` as part of your submission that should contain the following files:

- Your final `accel_audio.hex`
- Your final `et4351.phys.sdf`
- Your final `et4351.phys.v`

Note that the full grading rubric will be shared next week.

2.2 Reproduce the Baseline

Before reproducing the baseline, we strongly recommend you to delete your previous lab folders since you only have 8 GB of disk quota (or alternatively, delete all old VCD files by running this command `find . -type f -name "*.vcd" -delete` from the directory in which all your labs are stored). The baseline design will occupy around 1.5 GB of disk space.

Run the following commands to reproduce the baseline. Expect long runtime around 1-2 hours:

```
1 cd ~
2 cp -r /data/labs/2026/project ~/
3 cd project
4 source run_all.sh
```

Please carefully inspect the `run_all.sh` script, which shows all the necessary steps for your own design.

3 Documentation

Please find below the paths on the server to various documentation for the used tools (Genus, Innovus and QuestaSim).

3.1 Genus

- User guide:
`/eda/cadence/2021-22/RHELx86/GENUS_21.10.000/doc/genus_user/genus_user.pdf`
- Messages:
`/eda/cadence/2021-22/RHELx86/GENUS_21.10.000/doc/genus_messages/genus_messages.pdf`
- Low power (including clock-gating):
`/eda/cadence/2021-22/RHELx86/GENUS_21.10.000/doc/genus_lowpower/genus_lowpower.pdf`

3.2 Innovus

- Flow setup:
`/eda/cadence/2021-22/RHELx86/INNOVUS_21.11.000/doc/flowSetup/flowSetup.pdf`
- User guide (UG):
`/eda/cadence/2022-23/RHELx86/INNOVUSEXPORT_21.35.000/doc/innovusUG/innovusUG.pdf`
- Text Command reference (TCR):
`/eda/cadence/2022-23/RHELx86/INNOVUSEXPORT_21.35.000/doc/innovusTCR/innovusTCR.pdf`

3.3 QuestaSim

- User guide:

`/eda/mentor/2020-21/RHELx86/QUESTA-CORE-PRIME.2020.4/questasim/docs/pdffdocs/
questa_sim_user.pdf`